gateway-admin

The staff edge of the announce marketplace: how to run it, sign in, moderate, administer, extend and deploy it

Module **gateway-admin** (port 8081), branch **feat/gateway-admin**, 2026-09-26. Design: *doc/frontend-edges.md* sections 2, 4, 4a, 6 and 7a. Companion documents: *doc/announce-management-context.md* section 10 (staff moderation), *db-reactive/sql/announce_auth.sql*.

Contents

- 1. What it is and how the pieces fit
- 2. Getting started in ten minutes
- 3. Configuration reference
- 4. Identity: sign-in, tokens, roles and permissions
- 5. Routes and the edge-hosted admin API
- 6. Moderating announces (the announce-management side)
- 7. Using the admin SPA
- 8. Applying it: deployment, extension, coordination
- 9. Tests, checklist and troubleshooting

1. What it is and how the pieces fit

gateway-admin is one Spring Boot 3.5 / Spring Cloud Gateway (WebFlux) application that plays three roles at once, exactly like litemall's admin gateway:

- **API gateway.** Explicit, ordered routes forward */api/*** calls to the services (announce-management, order, messaging) without StripPrefix. Every proxied request loses the staff token and cookies and gains *X-User-Id*, *X-User-Type=ADMIN*, *X-User-Roles* and a signed *X-Service-Token*.
- **Identity provider for staff.** */auth/login|refresh|logout|me* over the *announce_admin*, *announce_role* and *announce_permission* tables; RS256 access tokens (2 h) plus an httpOnly refresh cookie (30 days, single use).
- **Host of the admin SPA** *@announce/admin-web* (React 18, TypeScript, Redux Toolkit, react-bootstrap, i18next en/da/sv) served from *classpath:/static*, with HTML5 deep-link fallback.
- **Edge-hosted admin CRUD** for what no bounded context owns yet: members, administrators, roles, permissions, regions, dashboard counts and crawled-listing statistics, as reactive controllers over the db-reactive mappers.

Component	Where	Talks to
Routes	*src/main/resources/config/application.yml*	announce-management 8085, messaging 8086, order (Eureka or fixed URL)
Security	*security/*: SecurityConfig, PublicPaths, JwtAuthenticationManager, IdentityForwardingFilter	AdminJwtService
Auth	*auth/*: AuthController, AdminAccounts, RefreshTokens	MySQL announce_admin, announce_role, announce_permission, announce_refresh_token
JWT	*jwt/*: AdminJwtService (Nimbus), RsaKeys, JwtConfig, JwtKeyGuard	config/dev-admin-jwt-*.pem or env PEMs
Machine token	*machine/*: MachineTokens, MachineTokenRelayFilter	shared secret with the backends

Component	Where	Talks to
Edge CRUD	*web/admin/*: Edge*Controller, AdminApiExceptionHandler	db-reactive mappers incl. AdminDashboardMapper
SPA	*src/main/webapp* (webpack to target/classes/static)	the edge on the same origin

Request path of a moderation click in the SPA: browser → *POST /api/admin/announces/{id}/withdraw* with *Authorization: Bearer* → SpaHistoryFallbackFilter (not a page) → IdentityForwardingFilter (strips client X-User-*, injects the staff identity) → Spring Security (ROLE_ADMIN required on /api/**) → route *admin-announces* → MachineTokenRelayFilter (drops the JWT, adds X-Service-Token) → announce-management dispatches *ModerateWithdraw*.

2. Getting started in ten minutes

Prerequisites

- Java 21 (*/usr/lib/jvm/java-21-openjdk-amd64*; Maven uses it), MySQL with *announce_reactive_db* and user *announce* (run *db-reactive/sql/announce_reactive_db.sql* or, on an existing database, *announce_auth.sql* plus the two moderation columns of *announce_table.sql*).
- *docker/cqrs* up (EventSourcingDB on 3000, OCS indexer 8535, searcher 8534) for announce-management.
- Node is **not** required to build the edge: the *webapp* Maven profile downloads node v20.16.0 and npm 10.8.1 into *target/*. Use *-P '!webapp'* to skip the SPA build.

Build

```
# once per checkout: parent POM
mvn -N install
# edge without the SPA (fast, no node)
mvn -pl db-reactive,gateway-admin -P '!webapp' -DskipTests package
# edge with the SPA embedded (downloads node, npm ci, webpack)
mvn -pl db-reactive,gateway-admin -P webapp -DskipTests package
unzip -l gateway-admin/target/gateway-admin-0.0.1-SNAPSHOT.jar | grep static/index.html
# announce-management (the moderation endpoint lives there)
mvn -pl db-reactive,announce-management -P '!webapp' -DskipTests package
```

Build with *-pl db-reactive,<module>*: several worktrees share *~/.m2*, and another session may have installed a db-reactive SNAPSHOT without the columns and mappers this module needs. The reactor build resolves db-reactive from source.

Run locally

```
JAVA=/usr/lib/jvm/java-21-openjdk-amd64/bin/java
$JAVA -jar announce-management/target/announce-management-0.0.1-SNAPSHOT.jar        # 8085
$JAVA -jar gateway-admin/target/gateway-admin-0.0.1-SNAPSHOT.jar --spring.profiles.active=dev   # 8081
# then open http://localhost:8081/  and sign in with the seed super administrator admin123 / admin123
```

The *dev* profile (default) points the routes at *http://localhost:8085* (announce-management) and *http://localhost:8086* (messaging) because those services do not register with Eureka yet, signs tokens with the committed development key pair (a WARN line reminds you), and sets the refresh cookie without the Secure flag.

First calls

```
curl -s -c cookies.txt -H 'Content-Type: application/json' \
  -d '{"username":"admin123","password":"admin123"}' http://localhost:8081/auth/login
# -> {"accessToken":"eyJ...","tokenType":"Bearer","expiresIn":7200,"admin":{"id":1,"username":"admin123",...,"rol
TOKEN=...   # the accessToken
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8081/api/admin/dashboard?days=7
curl -s -b cookies.txt -X POST http://localhost:8081/auth/refresh      # rotates the cookie, new access token
curl -s -b cookies.txt -X POST http://localhost:8081/auth/logout       # 204, cookie cleared, token revoked
```

3. Configuration reference

All keys live in *gateway-admin/src/main/resources/config/application.yml*; environment variables override them.

Key	Default	Meaning
server.port	8081	The edge port.
announce.routes.announce-management / order / messaging	lb://<service> (dev: http://localhost:8085 / 8086)	Target URI of each route group. Env: ANNOUNCE_ROUTES_ANNOUNCE_MANAGEMENT etc. Use lb:// with Eureka, http:// for a fixed host.
eureka.client.enabled	false (ANNOUNCE_EUREKA_ENABLED)	Registration and lb:// resolution. Off until the services register.
announce.jwt.issuer / audience	announce-admin	Checked on every token.
announce.jwt.access-ttl	PT2H	Access token lifetime.
announce.jwt.private-key-pem / public-key-pem	blank (ANNOUNCE_ADMIN_JWT_PRIVATE_KEY_PEM / _PUBLIC_KEY_PEM)	PKCS#8 private and X.509 public key PEM. Blank selects config/dev-admin-jwt-*.pem; the prod profile refuses blank.
announce.refresh.ttl	P30D	Refresh token lifetime.
announce.refresh.cookie-name	announce_admin_refresh	httpOnly cookie, Path=/auth, SameSite=Strict.
announce.refresh.cookie-secure	false (ANNOUNCE_COOKIE_SECURE); prod: true	Secure flag; needs TLS in front.
announce.machine.secret	dev-service-secret-change-me (ANNOUNCE_SERVICE_SECRET)	HMAC key of X-Service-Token; must equal the backends' value.
announce.machine.issuer	gateway-admin	First segment of the service token.
spring.r2dbc.mybatis.r2dbc-url / username / password	r2dbc:mysql://localhost:3306/announce_reactive_db, announce (ANNOUNCE_R2DBC_URL, ANNOUNCE_DB_USER, ANNOUNCE_DB_PASSWORD)	Auth store and edge CRUD connection (reactive-mybatis-support keys).

Generating production keys

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out admin-jwt-private.pem
openssl pkey -in admin-jwt-private.pem -pubout -out admin-jwt-public.pem
export ANNOUNCE_ADMIN_JWT_PRIVATE_KEY_PEM="$(cat admin-jwt-private.pem)"
export ANNOUNCE_ADMIN_JWT_PUBLIC_KEY_PEM="$(cat admin-jwt-public.pem)"
export ANNOUNCE_SERVICE_SECRET="$(openssl rand -hex 32)"
```

Profiles: **dev** (default) fixed local routes and debug logging; **prod** secure cookie, health details hidden, forwarded headers honoured, and *JwtKeyGuard* aborts start-up when the key PEMs are blank. Never run prod with the committed development keys: anyone with the repository could mint staff tokens.

4. Identity: sign-in, tokens, roles and permissions

Sign-in flow

- **POST /auth/login** {username, password}: BCrypt check against *announce_admin.password* (off the event loop), last login time and IP recorded, roles resolved, access token in the JSON body, refresh token in the cookie. Wrong credentials, unknown or deleted account and an account without an enabled role all answer the same 401 problem *urn:announce:auth:bad-credentials*.
- **POST /auth/refresh** (cookie only): the presented token is looked up by SHA-256 hash, must be unrevoked and unexpired; it is revoked and a new one issued (rotation), the principal is reloaded from the database so a deleted account or a removed role ends the session. A reused or unknown token is 401 *invalid-refresh-token* and the cookie is cleared.
- **POST /auth/logout**: revokes the cookie's token, clears the cookie, 204. Access tokens stay valid until they expire (stateless); the SPA drops its copy.

- **GET /auth/me**: the identity from the token (no database call).

Token contents

```
{ "iss": "announce-admin", "aud": "announce-admin", "sub": "1", "jti": "...", "iat": ..., "exp": ... (+2h),
  "typ": "ADMIN", "uid": 1, "username": "admin123", "avatar": "...",
  "roles": ["ROLE_ADMIN", "ROLE_SUPER"], "roleIds": [1] }
```

Roles to authorities

announce_admin.role_ids is a JSON array of *announce_role* ids. At sign-in and at every refresh the edge keeps the roles that are enabled and not deleted:

- **ROLE_ADMIN**: the account has at least one such role. Required for everything under */api/*** and for */auth/me*.
- **ROLE_SUPER**: one of those roles holds the permission string ***** in *announce_permission*. Required for */api/admin/admins/***, */api/admin/roles/*** and */api/admin/permissions/***.
- No enabled role: the account cannot sign in (401).

To make an administrator a super administrator: give one of their roles the permission * (roles screen, or *PUT /api/admin/roles/{id}/permissions {"permissions":["*"]}*). The change applies at the next sign-in or refresh; existing access tokens keep their claims until they expire (at most 2 h).

Permission catalogue

GET /api/admin/permissions lists what a role may hold. Only * is enforced today; the other codes describe intent and are stored so a later step can gate routes by permission:

Code	Meaning
*	Super administrator
admin:announce:moderate	List every announce, withdraw and expire with a reason
admin:catalog:manage	Categories per kind and brands
admin:listing:manage	Crawled listings: import, reindex, statistics
admin:member:manage	Members: list, search, enable or disable
admin:region:manage	Regions
admin:order:manage	Auctions, orders, disputes and payouts
admin:support:manage	Support queue and bulletins
admin:dashboard:read	Dashboard counts

What the backends receive

Header	Value	Note
X-User-Id	the admin id	Only when the bearer token verifies and is of type ADMIN. Any client-supplied value is removed first.
X-User-Type	ADMIN	The customer edge sends MEMBER.
X-User-Roles	ROLE_ADMIN[,ROLE_SUPER]	Comma separated authorities.
X-Service-Token	gateway-admin.<epoch>.<hmac-sha256>	HMAC over issuer.epoch with announce.machine.secret; backends verify issuer, age and signature in a later step (MachineTokens.verify shows how).
Authorization, Cookie	removed	The staff JWT and the refresh cookie never leave the edge.

5. Routes and the edge-hosted admin API

Proxied routes (declaration order, no StripPrefix)

Path	Target	Who
/api/admin/listings/**	announce-management (import, reindex)	ROLE_ADMIN
/api/admin/announces/**	announce-management (AnnounceAdminController)	ROLE_ADMIN
/api/categories/**, /api/brands/**	announce-management (reads and writes)	ROLE_ADMIN
/api/admin/orders/**, /bids/**, /payouts/**, /auctions/**	order	ROLE_ADMIN
/api/admin/conversations/**, /api/admin/bulletins/**	messaging	ROLE_ADMIN

Local controllers win over routes because Spring's request-mapping handler (order 0) is consulted before the gateway's route handler (order 1); that is how */api/admin/users* and friends are served by the edge although */api/*** is otherwise proxied.

Edge-hosted endpoints

REST-shaped JSON; lists answer *{items, page, size, total}* (page is 1-based, size at most 100); errors are RFC 7807 problems with a *code* on conflicts.

Endpoint	Body / query	Notes
GET /api/admin/dashboard?days=30		announces[{kind,status,total}], listings[{source,status,total}], members, admins, categories, brands, listingClicks, newMembersPerDay, newAnnouncesPerDay
GET /api/admin/listing-stats?limit=10		counts per source and status, clicks, recentGone, topClicked (with url)
GET /api/admin/users?q&status;&page;&size; GET /{id}; PUT /{id}/status	{status: 0|1|2}	0 active, 1 disabled, 2 deregistered; q matches username, nickname, mobile
GET /api/admin/admins?q; GET/{id}; POST; PUT /{id}; DELETE /{id}	{username, password, avatar, roleIds}	SUPER only. Username 3-63 [A-Za-z0-9_.-], password ≥ 6, BCrypt on create and on an explicit change, never returned. 409 username-taken, self-delete
GET /api/admin/roles?q; GET /all; GET/{id}; POST; PUT /{id}; DELETE /{id}	{name, description, enabled}	SUPER only. 409 role-name-taken, role-in-use (still assigned)
GET /api/admin/roles/{id}/permissions; PUT	{permissions: [codes]}	Replaces the set; unknown code is 400
GET /api/admin/permissions		The catalogue of section 4
GET /api/admin/regions?pid&type;&q; GET/{id}; POST; PUT /{id}; DELETE /{id}	{pid, name, type 1|2|3, code}	Physical delete; 409 region-has-children

6. Moderating announces (the announce-management side)

Sellers withdraw their own announces and the scheduler expires overdue ones; neither path lets staff act on somebody else's announce. Two commands were added to the Announce aggregate for that, reached through *AnnounceAdminController* at */api/admin/announces*, which the edge routes to announce-management with the moderator's id in *X-User-Id*.

Endpoint	Effect	Rules
GET /api/admin/announces?kind&status;&seller;&region;&category;&saleType;&q;&page;&size;&sort;	Every announce of every seller in every status (the public listing forces PUBLISHED)	sort: newest, oldest, price_asc, price_desc, expiring
GET /api/admin/announces/{id}	The announce in any status with its kind block	404 when unknown

Endpoint	Effect	Rules
POST /api/admin/announces/{id}/withdraw {reason}	ModerateWithdraw → AnnounceWithdrawn(at, moderatedBy, reason) → WITHDRAWN; removed from the search index	PUBLISHED only (409 not-published); reason non-blank, ≤ 255 chars (400)
POST /api/admin/announces/{id}/expire {reason}	ModerateExpire → AnnounceExpired(at, moderatedBy, reason) → EXPIRED, whatever the expiry date	same

The read model records *announce.moderated_by* and *announce.moderation_reason* on every status transition: set by a staff action, cleared by a seller or scheduler transition, so a republished announce shows no stale reason. The admin listing exposes them as *moderatedBy* and *moderationReason*; the seller can republish a moderated announce like any withdrawn one. An optional *X-Command-Id* header makes a retry idempotent.

Seed data caveat: the rows migrated from litemall (ids such as 1006002) exist only in the read model and have no event stream, so moderating them answers 404 *urn:announce:cqrs:subject-not-found*. Only announces posted through the API can be moderated.

7. Using the admin SPA

Open *http://localhost:8081/*. The shell has a sidebar per module, a language switch (en, da, sv; remembered in the browser) and the signed-in administrator with a sign-out entry. The access token lives only in memory; on start the SPA calls */auth/refresh* once to restore the session from the cookie, and on any 401 it refreshes once and retries before sending you to the login page.

Screen	Path	What you do there
Login	/login	Username and password; errors from the problem detail
Dashboard	/	Counts (members, admins, categories, brands, listing clicks), announces per kind and status (click through to the filtered list), listings per source and status, two per-day charts with a 7/14/30/90-day window
Categories	/categories	One tab per kind; tree; create a root, add a child, rename or describe, move under another parent, hide and show
Brands	/brands	Paged list, create, edit, hide
Announces	/announces, /announces/:id	Filters kind, status, seller, region, text, sort (kept in the URL); kind badges and per-kind facts; moderator and reason when present; Withdraw and Expire dialogs that require a reason; detail with the kind block
Crawled listings	/listings	Counts per source and status, most clicked (link to the source site), recently gone; Import (file path and source) with its report; Reindex
Members	/users, /users/:id	Search, status filter, detail, set active / disabled / deregistered with confirmation
Administrators	/admins (super only)	Create with roles from the picker, edit, optional new password, delete; conflicts shown by code
Roles	/roles (super only)	Create, edit, enable; permission checkboxes from the catalogue; delete refused while assigned
Regions	/regions	Breadcrumb drill-down from the roots, search, create, edit, delete
Auctions, Orders, Payouts	/auctions, /orders, /payouts (flag trade)	Phase 2 screens against MSW mocks shaped by doc/order-context.md sections 8 and 9; a banner says the data is mocked
Support, Bulletins	/support, /bulletins (flag chat)	WAITING queue with Join, bulletin posting; mocked as above

Feature flags

Phase 2 entries appear only when a flag is on. Switch them without a rebuild by opening *http://localhost:8081/?features=trade,chat* (persisted in the browser) or by setting *localStorage['announce.features']* to *{"trade":true,"chat":true}*. Mocks start only outside production builds or

when *mocks* is on as well.

Front-end development

```
cd gateway-admin/src/main/webapp
npm ci
npm run webapp:dev     # webpack-dev-server on http://localhost:9001, proxies /api and /auth to the edge on 8081
npm test               # jest: auth slice, refresh-and-retry, filters, category validation, moderation dialog, ro
npm run i18n:check     # every key present in en, da and sv, no empty values, no t('...') literal missing in en
npm run typecheck && npm run build   # webpack production build into target/classes/static
```

Every visible string goes through *t()*; add a key to all three files under *app/i18n/{en,da,sv}/* or the check fails. Typed API clients live in *app/shared/api* and mirror the controllers' DTOs by hand; when a controller changes, change the client.

8. Applying it: deployment, extension, coordination

Docker

```
mvn -pl db-reactive,gateway-admin -P webapp -DskipTests package
docker build -t announce/gateway-admin gateway-admin
# or through docker/docker-compose.yml (service gateway-admin), which requires:
#   ANNOUNCE_DB_PASSWORD, ANNOUNCE_ADMIN_JWT_PRIVATE_KEY_PEM, ANNOUNCE_ADMIN_JWT_PUBLIC_KEY_PEM, ANNOUNCE_SERVICE
docker compose -f docker/docker-compose.yml up gateway-admin
```

The image runs the *prod* profile: real keys and a real machine secret are mandatory, the refresh cookie is Secure (terminate TLS in front and forward the standard headers; *server.forward-headers-strategy=framework* is set), health details are hidden. Point *ANNOUNCE_ROUTES_** at the service hostnames or enable Eureka.

Adding a proxied admin route

- Add a route under *spring.cloud.gateway.server.webflux.routes* with *uri: ${announce.routes.<service>}* and its Path predicates; keep specific prefixes before broader ones.
- It is already protected: */api/*** needs ROLE_ADMIN. Add the path to *SecurityConfig.SUPER_PATHS* if only super administrators may use it.
- Add a typed client in *app/shared/api*, a module under *app/modules*, a sidebar entry and i18n keys in the three languages.
- Add a route test in *EdgeRoutesTest* (MockWebServer receives the call with the identity headers).

Adding an edge-hosted controller

- Put it in *web/admin*, inject the db-reactive mapper, return *PageResponse* for lists and throw *EdgeErrors.NotFound* / *EdgeErrors.Conflict(code, message)*; *AdminApiExceptionHandler* turns them into problems.
- Do blocking work (BCrypt) on *Schedulers.boundedElastic()*; everything else is reactive.
- Test it against the live database like *EdgeAdminControllersTest*: create rows with a unique prefix and delete them in *@AfterAll*.

Adding a permission

Append a *Permission(code, description)* to *PermissionCatalogue.ALL*; the roles screen shows it and *PUT /permissions* accepts it. Enforcing it means mapping the code to authorities in *AdminAccounts.principalOf* and adding a matcher in *SecurityConfig*.

Coordinating with gateway-api

Both edges share the design and duplicate the security package on purpose (extraction into a shared module is a later step). They also share the table *announce_refresh_token* (id is a UUID chosen by the edge,

user_type MEMBER or ADMIN, hashed token, single-use rotation), the file *db-reactive/sql/announce_auth.sql*, the root *pom.xml* module list and *docker/docker-compose.yml*. When merging the second worktree, keep one copy of the DDL and mapper and both compose services. Build with *-pl db-reactive,<module>* while both worktrees are active, because they install different db-reactive SNAPSHOTs into the shared *~/.m2*.

9. Tests, checklist and troubleshooting

Automated

```
mvn -pl db-reactive,gateway-admin -P '!webapp' test
#   EdgeRoutesTest (10): 401 without token, 403 for a member token, admin token -> X-User-Id/Type/Roles and
#     X-Service-Token downstream, spoofed headers dropped, JWT and cookie not relayed, super-only paths,
#     problem pass-through, SPA fallback and 404 for a missing asset, public /auth surface
#   AuthFlowTest (4, live MySQL): login, /auth/me, refresh rotation, reuse refused, logout revokes
#   EdgeAdminControllersTest (5, live MySQL): roles + permissions, admins (BCrypt, conflicts), regions,
#     members, dashboard, listing stats
mvn -pl db-reactive,announce-management -P '!webapp' test
#   AnnounceTest (moderation rules), AnnounceAdminControllerTest (slice), AnnounceKindsEndToEndTest
#   (Testcontainers: staff withdraws and force-expires, reason reaches the read model, republish clears it)
cd gateway-admin/src/main/webapp && npm test && npm run i18n:check
```

Manual checklist (doc/frontend-edges.md section 7a, executed 2026-09-26)

- Sign in as admin123: token with ROLE_ADMIN and ROLE_SUPER, refresh cookie on /auth.
- Create a category of each kind through the edge: three 201s, visible in the kind trees.
- Post and publish an announce as a seller, withdraw it as staff with a reason: WITHDRAWN, moderator and reason in the admin listing.
- Trigger a listings import of *dba_cars.json*: 54 seen, all unchanged; reindex 200; statistics list the most clicked listing.
- Create a role with two permissions and an administrator holding it: the new administrator signs in with ROLE_ADMIN only, is refused on /api/admin/admins (403) and allowed on the dashboard.
- The SPA shell answers / and deep links; assets load; /api without a token is a 401 problem.

Troubleshooting

Symptom	Cause and fix
401 urn:announce:edge:unauthenticated on /api	No or malformed Authorization header. The SPA refreshes once; if the cookie is gone, sign in again.
401 invalid-token	Signature, issuer, audience or expiry failed: the edge was restarted with other keys, or the token is from the customer edge.
403 forbidden on /api/admin/admins	The account lacks ROLE_SUPER: give one of its roles the permission *, then refresh.
401 bad-credentials for a known account	Password wrong, account deleted, or none of its roles is enabled.
404 subject-not-found when moderating	Seed row without an event stream (section 6).
503 from a route	Target unreachable: check announce.routes.* (dev uses fixed ports) or Eureka registration.
Jar has no static/index.html	Built with -P '!webapp', or the checksum skip fired after a compile-only run: rm gateway-admin/target/checksums.csv.old and package again with -P webapp.
ClassNotFound / NoSuchMethod on db-reactive classes	Another worktree installed a different db-reactive SNAPSHOT into ~/.m2: build with -pl db-reactive,<module>.
Start-up fails under prod	JwtKeyGuard: set the two PEM variables; MachineTokenProperties: set ANNOUNCE_SERVICE_SECRET.
announce-management /actuator/health DOWN	A health indicator of an unused binder; the API works. Not caused by the edge.